

Course Name: Deep Learning

Batch: 2026-2027

Course Code: 22AIE304

Lab 1: Perceptron Learning Implementation

Learning Objective (CO1)
 Implement the Perceptron learning algorithm from scratch using NumPy to understand the fundamental building blocks of neural networks. You will focus on the weight update rule and the step activation function for binary classification.

Step-by-Step Code Logic

Before implementing the full training loop, define the mathematical logic for the Perceptron's core components. Use the following notation: w for weights, b for bias, x for input, and η for learning rate.

Component	Implementation Logic / Formula
Activation Function (Heaviside Step)	$z = w \cdot x + b$ $f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$
Weight Update Rule (Δw)	$\Delta w = \eta(y - \hat{y})x$ $w(\text{new}) = w(\text{old}) + \eta(y - \hat{y})x$ $b(\text{new}) = b(\text{old}) + \eta(y - \hat{y})$
Prediction Logic ($\hat{y}$)	$\hat{y} = f(w \cdot x + b)$

Training Loop Implementation

Complete the Python template below to implement the Perceptron training process for a linearly separable dataset (e.g., AND gate).

```
python
import numpy as np

def train_perceptron(X, y, lr=0.1, epochs=10):
    weights = np.zeros(X.shape[1])
    bias = 0

    for _ in range(epochs):
        for i in range(len(X)):
```

- **Calculate Linear Combination**

```
linear_output = np.dot(X[i], weights) + bias
```

- **Apply Activation Function**

```
y_pred = 1 if linear_output >= 0 else 0
```

- **Compute Update (Error * Learning Rate)**

```
update = lr * (y[i] - y_pred)
```

- **Update Weights and Bias**

```
weights += update * X[i]
```

```
bias += update
```

```
return weights, bias
```

Output Analysis

Execute your implementation using a linearly separable dataset (AND or OR gate).
Document your findings below.

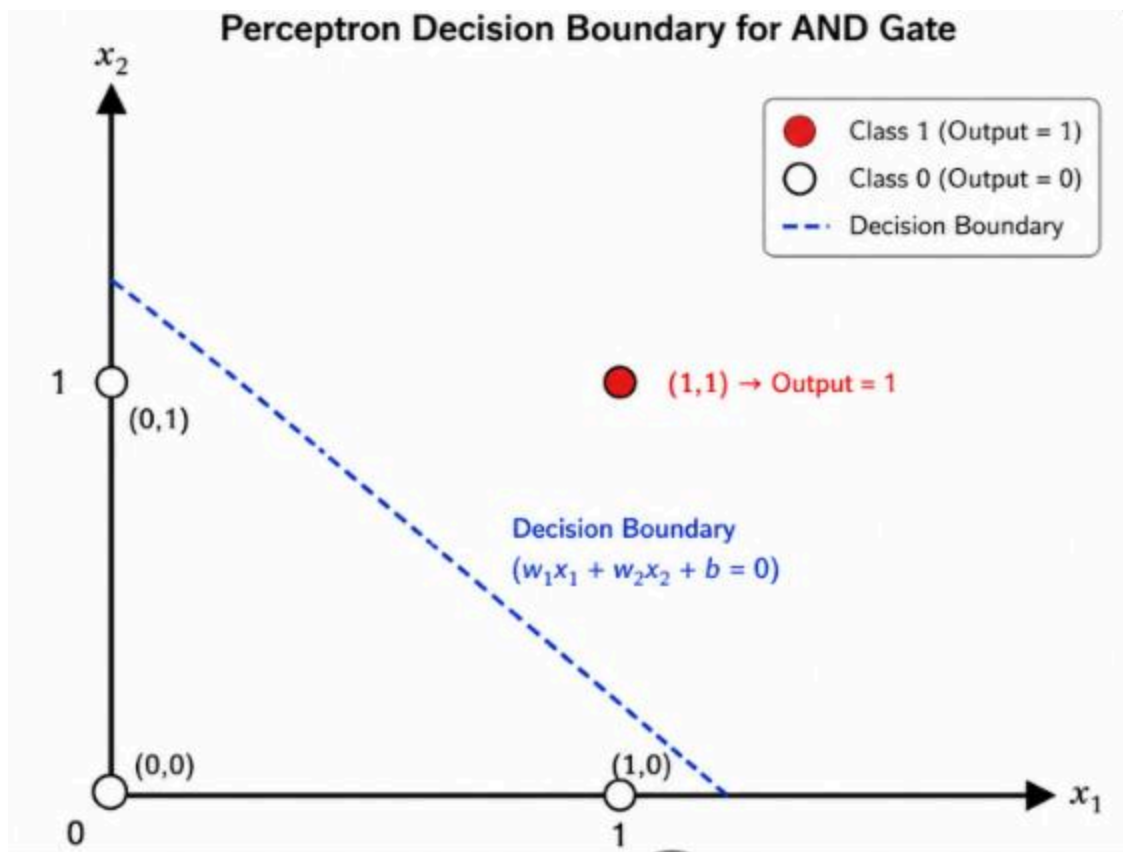
Metric/Observation	Experimental Results
Dataset Used (AND/OR)	AND Gate
Final Weights (w_1, w_2)	(0.2, 0.1)
Final Bias (b)	-0.2
Decision Boundary Equation ($w_1x_1 + w_2x_2 + b = 0$)	$0.2x_1 + 0.1x_2 - 0.2 = 0$

Decision Boundary Visualization

Sketch or describe the resulting decision boundary relative to the data points. Explain if the Perceptron successfully converged and why.

The decision boundary obtained by the Perceptron is a straight line defined by the equation $w_1x_1 + w_2x_2 + b = 0$. This line separates the two classes of the AND gate dataset. The points **(0,0)**, **(0,1)**, and **(1,0)** lie on the output class **0** side of the boundary, while the point **(1,1)** lies on the output class **1** side. Since the AND gate is linearly separable, the Perceptron successfully converged after a few training epochs and correctly classified all training samples.

The Perceptron successfully converged because the AND gate is a **linearly separable dataset**. A single straight line is sufficient to separate the two output classes. According to the **Perceptron Convergence Theorem**, the algorithm is guaranteed to converge for any linearly separable dataset when an appropriate learning rate is used.



Critical Thinking: If you were to provide a non-linearly separable dataset like XOR to this implementation, what would happen to the weights during training? Explain based on the Perceptron Convergence Theorem.

If a non-linearly separable dataset such as the XOR gate is used, the Perceptron will not converge to a stable set of weights. During training, the weights and bias will continue to change because no single linear decision boundary can correctly classify all XOR data points. According to the **Perceptron Convergence Theorem**, a Perceptron is guaranteed to converge only when the training data is **linearly separable**. Since the XOR dataset is **not linearly separable**, the algorithm cannot find a solution and the training process fails to achieve 100% accuracy.